

A behavior-aware SLA-based framework for guaranteeing the security conformance of cloud service

Xiaochen LIU¹, Chunhe XIA^{1,2}, Tianbo WANG^{1,3}, Li ZHONG¹, Xiaojian LI (✉)²

- 1 Beijing Key Laboratory of Network Technology, School of Computer Science and Engineering, Beihang University, Beijing 100191, China
- 2 School of Computer Science and Information Technology, Guangxi Normal University, Guilin 541004, China
- 3 School of Cyber Science and Technology, Beihang University, Beijing 100191, China

© Higher Education Press and Springer-Verlag GmbH Germany, part of Springer Nature 2020

Abstract As cloud computing technology turning to mature, cloud services have become a trust-based service. Users' distrust of the security and performance of cloud services will hinder the rapid deployment and development of cloud services. So cloud service providers (CSPs) urgently need a way to prove that the infrastructure and the behavior of cloud services they provided can be trusted. The challenge here is how to construct a novel framework that can effectively verify the security conformance of cloud services, which focuses on fine-grained descriptions of cloud service behavior and security service level agreements (SLAs). In this paper, we propose a novel approach to verify cloud service security conformance, which reduces the description gap between the CSP and users through modeling cloud service behavior and security SLA, these models enable a systematic integration of security constraints and service behavior into cloud while using UPPAAL to check the performance and security conformance. The proposed approach is validated through case study and experiments with real cloud service based on OpenStack, which illustrates CloudSec approach effectiveness and can be applied on realistic cloud scenario.

Keywords security conformance, security SLA, cloud behavior modeling

1 Introduction

The maturity of cloud service technology effectively solves the problem of traditional limited and unexpandable service resources. However, the opacity of security cloud QoS is the biggest concern for users as using cloud services [1]. Many organizations are working on improving the security and privacy of cloud services in the hope of gaining user trust. Therefore, if the CSP can prove that the infrastructure and behavior of cloud service are in line with the user's requirements, it can enhance the user's confidence in implementing it. Therefore, in the security perspective of cloud services, how to effectively verify the security conformance of cloud services has become a hot topic [2]. Through the verification of cloud service conformance, users can judge the cloud service they use whether to meet their security and performance requirements. It helps improve cloud service quality and better for CPSs to optimize their services. The existing conformance judgment methods mainly focus on the QoS and service processes of cloud services. For example, the research [3] gives a method for judging the conformance of key performance indicators in cloud services. The research [4] presents the service process conformance verifying method of the cloud service, which is performed only for checking cloud service processes conformance, and it is impossible to determine the conformance of specific service behaviors

and status, so that complex functional SLA metrics cannot be judged. Therefore, the conformance verification technology based on model checking is proposed [5–10]. The main idea of this method is based on software engineering methods to detect whether the software system can meet certain specific properties. Researchers treat cloud services as a complex real-time system, and SLAs are seen as specific properties and verify cloud service conformance through real-time judge systems whether satisfy properties. By analyzing the existing research, these methods have some limitations, the main three reasons are listed as follows:

First, there is a lack of appropriate method to verify cloud conformance. The cloud infrastructures are mainly characterized by dynamicity and scalability, requiring the modeling process must be adjusted to dynamicity and reconstructed, we need to verify the state of cloud behavior. In [9], which build a probabilistic model of the business process and verify SLA compliance based on PRISM model checker. However, this approach using probabilistic is not suitable for cloud scenario, because cloud not only focuses on physical and virtual infrastructure behavior, but also concentrate on the cloud behavior conformance with SLA properties, rather than probabilistic model checking.

Second, there are lack of specific languages for modeling cloud service, traditional modeling language BPEL [6] focusing on the service process, it has ignored the details of underlying service mechanism and the transition between different service states. In [5] which based on Petri-net cannot express the interaction behavior in the cloud environment with exact constraints. The study [11] illustrates a specific language named Cloud# language, which can improve the transparency of the bottom of cloud service, but without describing the security or performance constraints.

Third, there is no standard SLA modeling language. Some related work focuses on the non-functional terms, such as performance metrics [12, 13], while only expresses numerical metrics, ignores the functional term, especially on security functional terms.

Therefore, in order to effectively solve the cloud service conformance verification, we need to solve the following problems: (1) how to describe the cloud service behavior in a fine-grained manner, and (2) how to describe the functional entries in the security SLA.

This paper presents a novel method based on model checking to solve the above problems. First, the method includes a new cloud service behavior modeling method and a security SLA description language specifically for cloud secu-

rity QoS. The cloud service modeling method is presented as CloudSec model can describe the state changing of the service itself, and also can describe the interaction between multiple cloud service behaviors. At the same time, it accurately describes the behavior of cloud services by adding security QoS constraints when describing the status changes. The security SLA description language involves performance metrics as well as functional metrics that can describe security entries. Then based on the above model, the model checking tool UPPAAL is used to verify the conformance of cloud service behavior and security SLA. The checking result helps users to understand whether the cloud security QoS meets their requirements. Finally, by establishing an OpenStack cloud platform to verify that this method can effectively solve the actual cloud service problem.

The paper is organized as follows: Section 2 introduces the related work of our research, Section 3 illustrates the problem of CloudSec approach solved. In Section 4, we proposes a cloud conformance checking approach, including the Cloud Behavior Model and the specific modeling language for Security SLA, while Section 5 relates to validating the method in case study and a real cloud scenario based on OpenStack, Section 6 is the conclusion and future work.

2 Related work

In recent years, various efforts have been devoted to the web service conformance checking based on SLA, which is surveyed in [5–10, 14–21], which can be classified into three types approach respectively as follows:

2.1 Approaches based on model checking

A conformance checking using a model checking thoughts is introduced in [7] and [8]. The work of [7] proposes a QoS prediction approach which combines runtime monitoring of the real system with model-checking on a parametrics system model, and then check the SLA conformance. Although this method also takes advantage of the model checking technique and extracting monitoring data in realtime, it would update the model more frequently and bring state explosion in the model. [8] proposes a model named ATOP, which derives a probabilistic model through abstracting description of a service activity diagram. However, this methodology can not adapt to the dynamic evaluation of cloud services because the original ATOP need designing for a long time, once the cloud service compositions changed, the original model would not be used to check current cloud service.

2.2 Approaches based on QoS values

[14] propose an approach for the service conformance checking problem that is both efficient and QoS-aware, considers numerical QoS metrics and maximizes QoS over the long-term, and results in a set of probabilistic service conformance results. [18] have studied the dynamic WS conformance checking problem in a dynamic environment, which aims to determine a subset of web services to be invoked at runtime so as to successfully orchestrate a composite web service satisfying SLA requirement. They have defined QoS metrics which related to a dynamic environment, used these values to measure the probability that a given state in the composite service will lead to successful execution, and also proposed an efficient method to validate it.

2.3 Approaches based on service process

An approach on conformance checking of service behavior is treated in [6], which converted event records of service into BPEL applied to derive two indicators of conformance, respectively are fitness and appropriateness. This approach can judge event trace but ignoring QoS. A collaborative method is proposed in [10] where conformance of cloud service is checked through service process standards based on experts' experience, although this method could be appropriate for conformance checking from, it is impractical in the general case where cloud services are various and dynamic developing into large scale. The work of [21] presents a conformance checking model of cloud infrastructure, where the authors exploit Markovian Arrival Process and a MAP/MAP/1 queuing model to predict QoS, and analyze the difference between prediction results and requirement specifications to check the conformance of cloud infrastructure.

Analyzing existing research, there is lack of an effective approach to make SLA conformance checking which adapts to cloud service dynamic developing scenario. In comparison to their work, we propose a more expressive cloud behavior model which adopt the finite state machine (FSM) with security constraints rather than another modeling approach such as Markov chain, thus our model can not only describe the numerical QoS constraints of each service operation, but also can specify the interactions between the appropriate service operations from different cloud service providers. Moreover, more desirable functional and unfunctional properties of the model can be specified in SecSLA than the previous work. We would apply the model checking techniques to automatically verify and analyze the conformance of cloud behavior using UPPAAL.

3 Motivation and conceptual model of cloud-sec approach

3.1 Conceptual model

In this section, we introduce some formal definitions related with cloud service conformance checking, and then clearly demonstrate what the solution is to this problem.

Definition 1 (Cloud service (CS)) A cloud service consists of a finite set of behaviors, denoted as $CS = \{atom_cs_1, atom_cs_2, \dots\}$, where each $atom_cs$ is related to a series of behaviors with a specific function. $atom_cs$ is defined by a tuple as $(PR, VR, csBeh, QoS)$, where $PR = \{PR^1, PR^2, \dots\}$ is a set of physical resources related with $atom_cs$, $VR = \{VR^1, VR^3, \dots\}$ is a set of virtual resources supplied to $atom_cs$, $csBeh = \{csBeh^1, csBeh^2, \dots\}$ is a set of cloud behaviors involved in $atom_cs$. $QoS = \{Q^1, Q^2, \dots\}$ is a group of cloud service QoS criteria.

Each cloud service can perform a set of behaviors, and combine others to perform a composite cloud service, which would have more complex functionality.

Definition 2 (Composite cloud service (CCS)) A composite cloud service is a structure $CCS = (\bigcup_{i=1}^n PR_i, \bigcup_{i=1}^n VR_i, csBeh_1 || csBeh_2 \dots || csBeh_n)$, expressing the set of different cloud service behavior.

Definition 3 (Security SLA (SecSLA)) A cloud service's security SLA, $SecSLA$, is a two-tuple $SecSLA = \{perfor, secur\}$, where $perfor = \{perfor^1, perfor^2, \dots\}$ is an performance metric set defined in $SecSLA$, and $secur = \{secur^1, secur^2, \dots\}$ is an security metric set describes cloud service's security requirements.

A security SLA $SecSLA = \{perfor, secur\}$ is specified by a user and cloud service provider to record the requirements about the quality of cloud service. It can be divided into two categories: performance and security. Performance-SLA denotes numerical metrics in cloud service, while Security-SLA correspond to data and virtual resources' security. Based on a typical cloud service used SLA [12], we would describe them as follows:

$$\begin{aligned} Perfor(CS) &= (Time(CS), Availability(CS), Cost(CS), \\ &\quad Memory(CS), Delay(CS), CPU(CS), \\ &\quad Bandwidth(CS)), \\ Secur(CS) &= (datasec(CS), VRsec(CS)). \end{aligned}$$

Definition 4 (Cloud service conformance checking problem)

A cloud service conformance checking problem, denoted as

$$CC = \text{checkcoformance}(CS, \text{SecSLA}),$$

where $CS = \langle cs_1, cs_2, \dots, cs_n \rangle$, $\text{SecSLA} = \langle \text{secsla}_1, \text{secsla}_2, \dots, \text{secsla}_n \rangle$.

A guaranteed term denoted by SecSLA can be evaluated with three different conformance checking values as output using a function $cc(q)$:

$$cc(q) = \{\text{Fullfilled}, \text{Violated}, \text{NotDetermined}\}.$$

- $cc(q_i) = \text{Fullfilled}$: If and only if the behaviors of the cloud service specified in the scope have been executed, the QoS of the cloud has been met SLO defined in SecSLA.
 $cc(q_i) = \text{Fullfilled}$ if $CS.q_i \subset *SecSLA$, where $\subset *$ means $CS.q_i \in CS.QoS$, and $CS.q_i$ can satisfy SecSLA.
- $cc(q_i) = \text{Violated}$: If and only if the behaviors of the cloud service specified in the scope have been executed, the QoS of the cloud has not been met SLO defined in SecSLA.
 $cc(q_i) = \text{Violated}$ if $CS.q_i \not\subset *SecSLA$, where $\not\subset *$ means $CS.q_i \in CS.QoS$, and $CS.q_i$ can not satisfy SecSLA.
- $cc(q_i) = \text{NotDetermined}$: If and only if the behaviors of the cloud service specified in the scope have not been executed.
 $cc(q_i) = \text{Violated}$ if $CS.Beh$ can not be traced as expect.

After providing a formalism way to evaluate the individual terms in SecSLA, we now focus on solving the conformance checking between all terms in SecSLA and cloud service.

According to the semantics of each cloud service, we define the following logic to make cloud service conformance checking with multiple guarantee SecSLA terms is evaluated as follows:

$$cc(CS) = cc(\text{All}_{i=1}^n(q_i)) = \begin{cases} \text{Fullfilled}, & \forall i \in [1, n], CS.q_i \subset *SecSLA, \\ \text{Violated}, & \exists i \in [1, n], CS.q_i \not\subset *SecSLA, \\ \text{NotDetermined}, & \exists i \in [1, n] : cc(q_i) = \text{NotDetermined} \\ & \wedge \text{not } \exists j \in [1, n] : CS.q_j \not\subset *SecSLA. \end{cases}$$

3.2 Motivation

In a real cloud scenario, users want to know the cloud service

behavior whether can meet their requirement, so we want to use an example to illustrate this problem.

Example 1 Assume a tour company needs a cloud storage service, which consists of storing data, processing data under strict safety requirements. If any customer's information is leaked, it will have a significant impact on the company's reputation. So verifying storage cloud behavior is important. However, traditional methods focus on exact numeric QoS values, ignore the behavior and complex security SLA. For example, that storage cloud service would supply the migration behavior if the state of vm1 is danger and vm2 is normal within 1ms as security SLA defined. That requirement cannot be adapted to traditional method because it contains two virtual machines and different states interaction.

Example 1 presented a trust-related cloud service scenario, that many potential users fear their sensitive data stored in the cloud by failsafe mechanisms, and other people can upload and backup their data. Therefore, in order to alleviate the user's concerns, the traditional method is to judge whether the cloud service is secure by monitoring the performance data, the trace of file history and log the access history, but it based on the log judgment has a delay and cannot be implemented in real time. There is a urgent need for an approach with real-time monitoring of cloud services, obtaining service behavior status and key QoS metrics for conformance verification.

So we can abstract this storage cloud as Fig. 1, which contains service entities, resource entities and QoS constraints. It has two types of resources entity, virtual resource entity, and physical entity, corresponding to the virtual machine and physical environment. Physical entity's state cannot be changed, but the virtual resource entity can be changed by interaction with cloud service. Cloud storage service involves controlling, receiving, monitoring and processing behaviors. The core is controlling behavior, which can obtain physical entities and virtual entities information through receiving behavior, and issue instructions to processing behavior and monitoring behavior.

The relationship between them is depicted in Fig. 1.

For the storage cloud service CS , it has five atom cloud services $\{acs_1, acs_2, acs_3, acs_4, acs_5\}$, the QoS criteria for each atom service and resource entity is showed in Fig. 1.

This QoS defined a certain level of performance, security, business and hardware must be satisfied. For example, the tour company wants to store sensitive data in the cloud, and require the data must be encrypted, backup automatic and accessed by privilege, but cannot be modified. Virtual resources must be protected by IDS, firewall, and vulnerability scan. So we can abstract this SecSLA as:

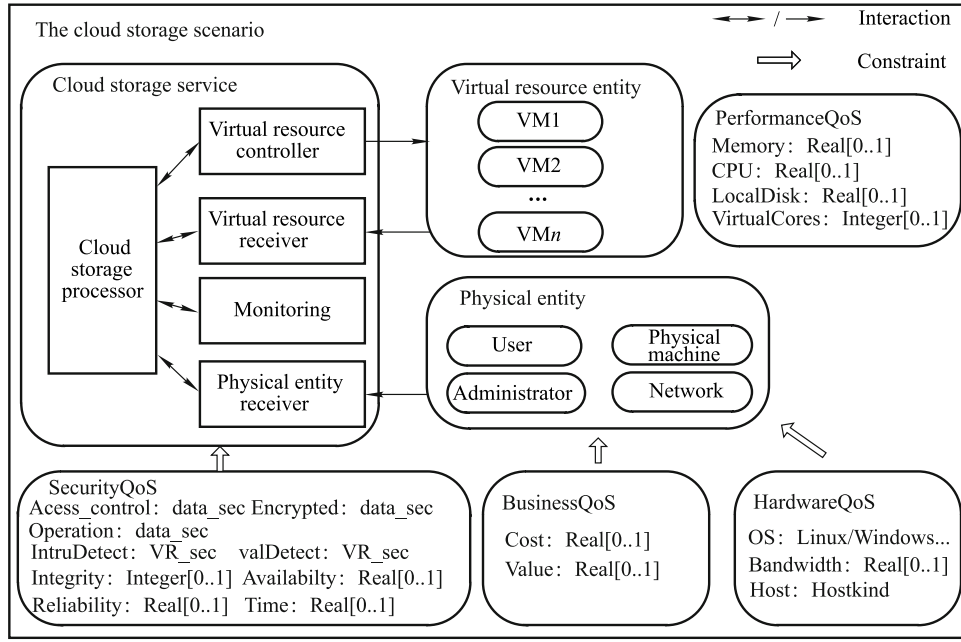


Fig. 1 The interaction relationship of storage cloud service

$$SecSLA = \{Psy_role, time, availabilty, reliability, data_VR_ \},$$

$$data_ = \{access_control, encrypted, operation\},$$

$$VR_ \equiv \{intruDetect, valDetect\}.$$

So our objective is to solve this conformance checking problem as to verify our cloud storage service whether satisfy some specific properties as a user in security SLA expected. In order to solve this problem, we define the CS Model and SecSLA model respectively and get the conformance checking results. Formally, we abstract this problem as follows:

$$\forall j \in [1, k] : \oplus cc(acs_j, q^j) = \{Fullfilled, Violated, NotDetermined\},$$

where $\forall i \in [1, n] : cc(q_i^j) = \{Fullfilled, Violated, NotDetermined\}.$

4 A novel approach of conformance checking

4.1 Overview approach

As shown in Fig. 2, our approach presented based on model checking considered cloud service behavior and security SLA.

Figure 2 introduces a new modeling method for cloud computing services called CloudBeh model. The first part focuses on cloud behavior model. The complex cloud's behavior involves a series of composite behavior entities through a tim-

ing automaton. The composite behavioral entity would be refined and divided into the smallest behavioral entities, called atom behavior entities, which are often used to describe fundamental cloud behavior. Each atom cloud behavior entity can send commands through channels defined in the timing automaton and deploy to each other to form a composite cloud behavior. It clearly describes the interaction between physical entities (such as servers and networks), virtual resource entities and atom cloud behavior entities, and the impact on cloud service quality. In order to characterize these entities, we must add some restrictions as transition conditions, such as the time, availability, cost, and security constraints discussed in the SLA. This modeling method will be consistent with the actual cloud computing application scenario.

Next, we propose the approach of cloud security SLA modeling called SecSLA model based on temporal logic. The traditional SLA description method only focuses on simple quantitative indicators, such as response time less than 0.5 seconds, cost less than 1000, and reliability greater than 99.5%. However, in actual cloud scenarios, the state and security requirements of cloud services are not only portrayed by numbers, but also need to be described by constraints of state transition between different cloud service states. Therefore, SecSLA focuses on how to describe performance and security metrics in cloud security SLAs, which are expressed by the constraints of cloud service entity behavior state transitions.

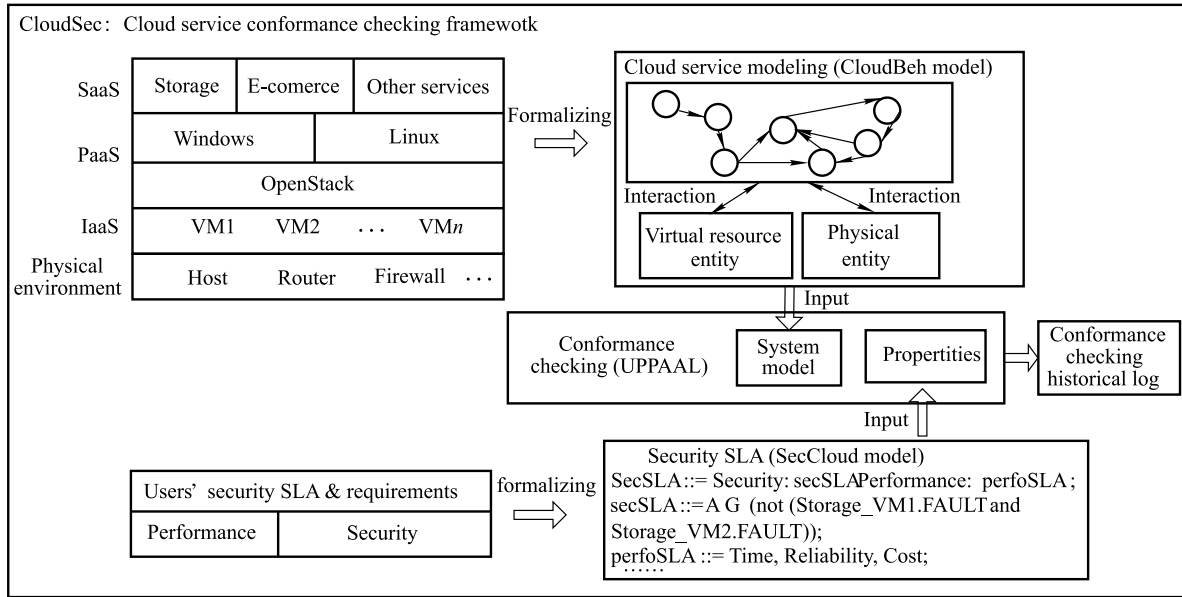


Fig. 2 Cloud service conformance checking approach

Finally, we check the conformance of the cloud service through the model checking tool UPPAAL. As an existing model checking tool, UPPAAL can detect whether a system with a complex interactive process can satisfy a series of specific property and give a judgment result. The model of cloud service behavior can be used as an input to the system in UPPAAL. The security SLAs can be treated as a series of properties to check whether the cloud service behavior satisfies the security SLA. This approach can improve the transparency and controllability of the underlying cloud services and perform conformance verification on the security SLAs of cloud services. We would demonstrate the effectiveness of our approach by conducting several experiments in the actual cloud platform based on OpenStack.

4.2 Building CloudSec model

This part focuses on cloud service behavior modeling methods, that model is fine-grained by depicting physical resource entities, virtual resource entities, atom service entities and the interaction process between them.

a. Modeling resource entity

Each resource entity should have the following basic characteristics:

- It has a unique identity to distinguish itself from other resources entities.
- It has certain physical or virtual attributes that are used to classify the categories of the resource entities. For

example, a physical resource entity has a physical machine ID and a physical machine performance; a virtual resource entity has a virtual machine ID, an operating system, and a virtual machine performance. If we assign a value to a resource entity which would become the state of the virtual resource.

- Both physical resource entities and virtual resource entities have communication capabilities, and their attribute values can be obtained by cloud services or other resource entities in an active or passive manner. The state of the virtual resource can be changed by interaction with the cloud service, while the physical resource cannot be changed.

(1) Physical resource entity

A physical resource entity refers to physical resource that provides a specific type of physical environment for a cloud service, such as physical computer, server, storage resource, security hardware, etc. Changes in the physical resource entity state would lead to the state changing of cloud service behavior entity and the virtual resource entity, but their own states are not affected by other entities.

Definition 5 (Physical resource entity) A physical entity is a five-tuple $(PREid, PREtype, PREAttr, PREOp, CS \ set)$, where

- $PREid$ is a physical entity identifier.
- $PREtype$ is a physical entity type indicator, which

stands for the role and privilege of each different physical resource entity.

- *PREAttr* is a set of physical entity related attributes, for example, user and administrator would have different privilege attributes, with each attribute treated as a state of the physical entity.
- *PREOp* is an operation set that physical entity can behave.
- *CS set* is a cloud service set, which interacts with the physical entity. PEs' behavior and information which can only be obtained by service without being changed.

(2) Virtual resources entity

A virtual resource entity refers to the virtual resource that participates in a specific cloud service, such as virtual computing resource, storage resource. Its state can be affected by physical resource entities and cloud service behavior entities, which states transition can also cause cloud behavior entity state changes.

Definition 6 (Virtual resource entity) A virtual resource entity is a four-tuple $(VREid, VREType, VREBeh, CS set)$, where

- *VREid* is virtual resource entity identifier.
- *VREType* is virtual resource entity type, such as computing resources and storage resources.
- $VREBeh \equiv (S, s_0, I_p, A_p, Trans)$ describes virtual resource entity behavior, S is a finite set of states, $s_0 \in S$ is the initial state, $I_p \subseteq Var$ is related information set of virtual resource entity, $A \subseteq Act$ is operation set, and $Trans \subseteq S \times \phi(I_p) \times A_p \times S \times R(I_p)$ is the edge set of state transition.
- *CS set* is a cloud service set, which interacts with the virtual resource entity. VRs' behavior and information which can not only be obtained by service, but also be controlled.

b. Modeling cloud service behavior

The previous described the example of cloud storage service illustrates the idea of cloud service modeling, a composite cloud service can be combined through a set of atom service behaviors with specific functions. It mainly includes describing the atom behavior and composite behavior of cloud services.

(1) Atom cloud service behavior entity

The description of atomic service behavior entity not only describes the state of the atom service behavior itself, but also presents its interaction with physical and virtual resource entities. For fine-grained characterization, a conceptual description of the timed automata is introduced [2].

Multiple concurrent time automata can constitute a timed automata network. Multiple time automata in the network would share some clock variables and data variables, but each has its own state.

Suppose there are n timed automata $TA_1, \dots, TA_n$, given an arbitrary natural number i , for $1 \leq i \leq n$, $TA_i = (S_i, s_i^0, Act_i, Clock_i, Var_i, I_i, E_i)$, where $I_i : S_i \rightarrow \Phi(Clock_i, Var_i)$ is the invariant of the state, $E_i \subseteq S_i \times \Phi(Clock_i, Var_i) \times Act_i \times R(Clock_i, Var_i) \times S_i$ is a set of directed edges.

The network composed of these n time automata is recorded as $N \equiv TA_1 || TA_2 || \dots || TA_n$. When $i=1$, the above definition represents a single timed automata.

The atom cloud service behavior entity refers to the most basic behavior in the cloud service and cannot be divided into more fine-grained behaviors, each atom cloud service behavior can be treated as a single timed automata. A composite cloud service behavior entity combines multiple atomic behaviors of a cloud service through interactions to implement more complex functions in the cloud service, which can be treated as a timed automata network.

Definition 7 (Atom cloud service behavior) An atom cloud service behavior is a structure $(CSid, PREset, VREset, CSBeh, QoS)$, where

- *CSid* is the identification of atom cloud service behavior.
- *PREset* is the set of physical resource entity interacted.
- *VREset* is the set of virtual resource entity interacted.
- *CSBeh* is a single timed automaton, which describes detailed cloud service behavior.
- *QoS* is a set of constraints describing service performance and security, which would affect *CSBeh*. Performance constraints mainly refer to numerical constraints such as time, cost, and reliability. Security constraints are mainly security functional terms such as integrity, privacy, and data security.

Through analyzing the actual cloud service environment, a typical cloud service behavior model including two types of physical resource entities, one virtual resource entity, and five types of atom service behavior entities, as shown in Fig. 3.

Physical resource entities are physical host entities and user entities respectively, and virtual resource entities are virtual machines which can provide specific cloud services. The virtual resource receiving atom behavior can obtain the state of the virtual machine within the clock constraint. The monitoring atom behavior can monitor the performance of composite cloud service, controlling atom behavior give instructions to other entities through analyzing the obtained virtual machine state and service performance, and perform related operations by processing atomic behavior.

Physical host entity: It refers to physical host performance attributes, such as CPU, memory, and localdisk, which can be communicated to other entities via a communication channel.

User entity: It refers to the user's role and privilege, which can send user request to other entities.

Virtual resource entity: It has five states, and time constraints and communication information from other entities would affect its state change. The $\{on\}$ state and the $\{off\}$ state are changed by the instruction on or off. When the virtual resource is in the $\{on\}$ state, through the basic security detection, such as intrusion detection and firewall detection, determines whether the virtual resource reaches the $\{normal\}$ state, and if the virtual resource passes the access control detection, encryption detection and integrity detection determine that the resource has reached a $\{safe\}$ state. If the resource is in the $\{normal\}$ state and cannot obtain return information exceeds the detection clock limit, it returns to the

$\{danger\}$ state.

Physical entity receiving atom behavior: It is used to receive physical entity information and status, such as CPU, memory, and localdisk, transfer them to virtual resource entities and other atom behavior. It can be formalized $(CS_PEreceiving, \{PRentity\}, \phi, PReceivingTA, \{QoS\})$.

Virtual resource entity receiving atom behavior: It is mainly used to receive the state of virtual resource entities such as $\{off, on, normal, safe, danger\}$ and transmit the obtained state information to other atomic behavior entities. It can be formalized $(CS_VRreceiving, \phi, \{VRentity\}, VRreceivingTA, \{QoS\})$.

Controlling atom behavior: It is the most central atomic service, it is responsible for making decisions and issuing instructions. It obtains state information through virtual resource entities and physical resource entities, and makes decisions to control state changes of other related entities through other atom behavior entity information. It can be formalized as $(CS_control, \phi, \{VRentity\}, ControlTA, \{QoS\})$.

Processing atom behavior: It is responsible for combining, analyzing, and processing all information from an individual entity, and sending the processing results to controlling atom behavior. If the user sends a request to modify or store their data, it justify the user's privilege, the $\{visit\}$ state which means the user only browse data but cannot modify. The $\{store\}$ state means the user has the privilege to modify data and determine whether to backup the data. It can be formalized $(CS_Processing, \phi, \phi, ProcessingTA, \{QoS\})$.

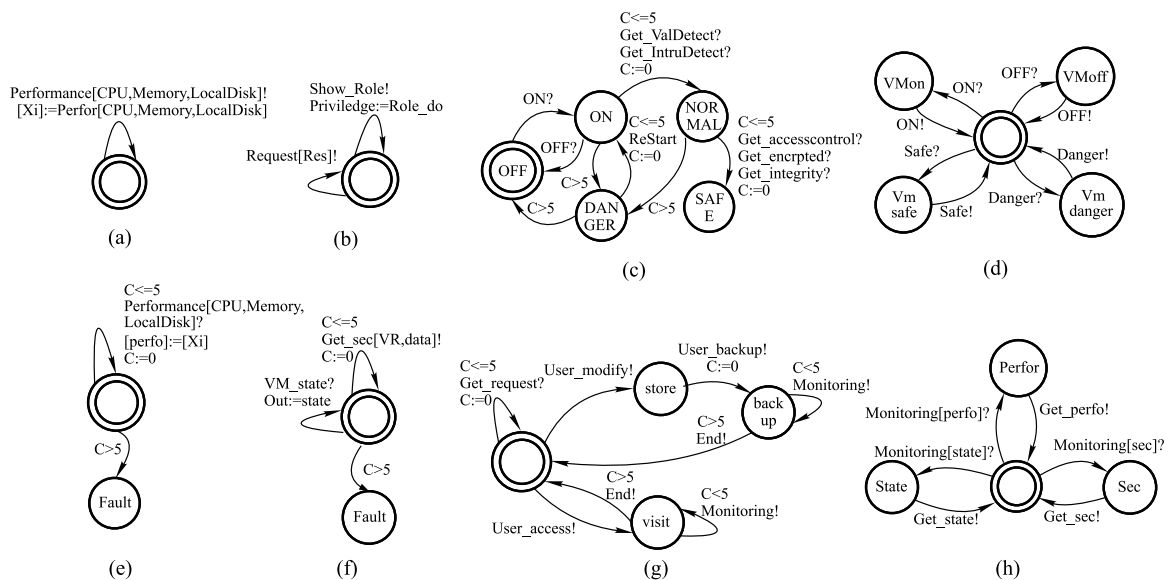


Fig. 3 The atom cloud behavior modeling. (a) Physical host entity; (b) user entity; (c) virtual resource entity; (d) control atom behavior; (e) physical entity receiving atom behavior; (f) virtual resource entity receiving atom behavior; (g) processing atom behavior; (h) monitoring atom behavior

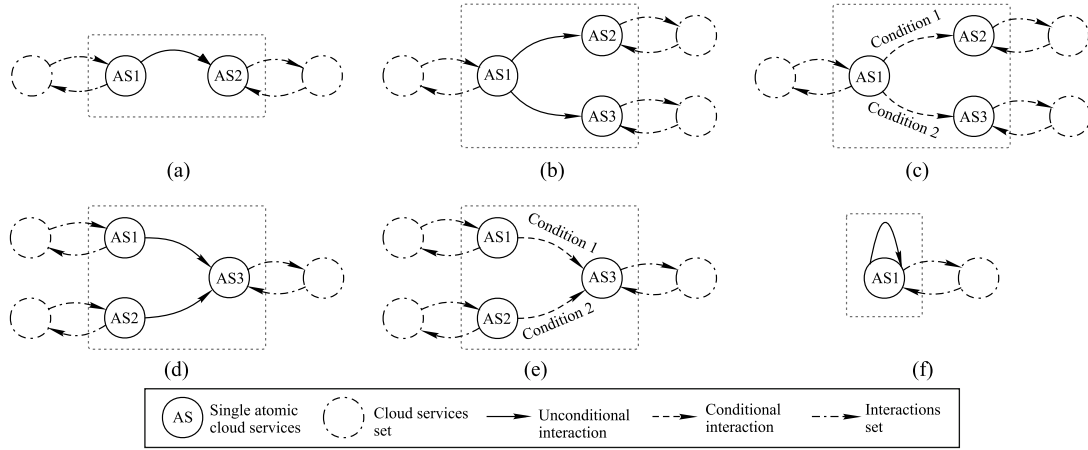


Fig. 4 Service composition pattern. (a) Sequential mode; (b) parallel mode; (c) exclusive selection mode; (d) synchronous mode; (e) selected merge mode; (f) loop mode

Monitoring atom behavior: It is used to monitor the security QoS and status of virtual resource entities, analyze the performance of the entire cloud service, sending the results to the controlling atom service. It can be formalized $(CS_Monitoring, \phi, \{VREntity\}, MonitoringTA, \{QoS\})$.

(2) Composite cloud service behavior

The complex service behavior that expresses the actual cloud scenario needs to be combined by the above resource entities and atomic service entities. The main six composite patterns are presented in Fig. 4 as Sequential mode, Parallel mode, Exclusive selection mode, Synchronous mode, Selected merge mode and Loop mode. In order to implement the composite cloud service behavior, it is necessary to establish a QoS-based interaction channel between each atom service behavior using a timed automata network.

Sequential mode is a composite service between two atomic services in sequence; Parallel mode is a composite service between the same atomic service and two other atomic services simultaneously; Exclusive selection mode is performed by the same atomic service and other two atomic services with conditional interaction, and conditions are mutually exclusive, that is, only one atomic AS2 or AS3 can be interacted at the same time to form a composite service; Synchronous mode is a composite service formed by two atomic services simultaneously interact with an atomic service; Selected merge mode is a composite service that two atomic services interact with the same atomic service at the same time, but the interaction is conditional and conditions are mutually exclusive; Loop mode is atomic services implement complex functions through their own state changes.

Definition 8 (Composite cloud service behavior) An

composite cloud service behavior is a structure $CS = (CSID, \bigcup_{i=1}^n PREset_i, \bigcup_{i=1}^n VREset_i, CBeh_1 || CBeh_2 \cdots || CBeh_n, \bigcup_{i=1}^n QoS_i)$ expressing the set of atomic cloud service behavior.

We can get all the composite cloud service pattern based on the above six patterns, the main idea of the aggregation of two cloud service behaviors as Algorithm 1:

Algorithm 1

Function *Aggregate Cloud Behavior* (*patternen*, *CloudBeh_CS1*, *CloudBeh_CS2*)

Instantiate a Composite cloud service behavior object as *CloudBeh*;

For every *behavior_cs1* in the *CloudBeh_CS1* do

 Instantiate a Behavior entity as *Behavior*;

 For every *behavior_cs2* in the *behavior_CS2* do

 If *behavior_cs1* and *behavior_cs2* are behavior of the same entities then

$Behavior = aggregateBeh (behavior_cs1, behavior_cs2, Pattern);$

 Add *Behavior* to Composite cloud service behavior;

 Delete *behavior_cs1* from *CloudBeh_CS1*;

 Delete *behavior_cs1* from *CloudBeh_CS2*;

 End If

 End For

 Add behavior of *CloudBeh_CS1* and *CloudBeh_CS2* to Composite cloud service behavior;

 Returns Composite cloud service behavior;

End Function

4.3 Building SecSLA model

This section focuses on the Security SLA modeling approach, which differs from the traditional SLA model that it focuses on characterizing security quality of service and performance metrics. For example, when the cloud service provides services to users, the SecSLA stipulates that the service reli-

bility is greater than 99% under the premise of ensuring the security of the virtual resources and privacy of data set.

In cloud computing infrastructure, Security function mainly involves Encryption, Data Recovery, Authentication, Fault Control, Disclosure Prevention. These function can not be depicted by traditional SLA language.

We specify the SecSLA by using TCTL, which is a clock extension of the temporal logic CTL and is appropriate to express the term of SecSLA. The syntax of TCTL is defined as follows:

$$\phi ::= a | \neg\phi | \phi \vee \phi | EX\phi | E[\phi U^{\leq k}\phi] | A[\phi U\phi],$$

where a is an atomic proposition, ϕ is state formulas, $X\phi$ denotes that ϕ is to be satisfied in the next state, and $\phi_1 U^{\leq k}\phi_2$ denotes that ϕ_2 should remain within k time units that ϕ_1 remain until that point. $\phi_1 U\phi_2$ denotes if ϕ_2 is satisfied at some point in the future and ϕ_1 remains until then.

For describing the security SLA for the actual cloud service environment, through analyzing the cloud service security requirements from existing standards, it mainly includes six types of security SLA terms, namely numerical QoS value, state reachability, time constraint, interactivity, resource state and system stability.

Numerical QoS value: it is used to describe numerical constraint entries in a security SLA, such as reliability greater than 99.5% and data migration time less than 0.5s.

State reachability: it is used to describe the reachability of a specific state of an entity involved in a cloud service process. For example, in a storage cloud service, whether the storage resource VM1 can reach the backup state.

Time constraint: it is used to describe the time constraint between state transitions. For example, the storage resource VM1 must turn to the backup state from the normal state within 0.1s.

Interactivity: it is used to describe the interaction between different entities. For example, if the storage resource VM1 status is danger, VM2 immediately performs data migration.

Resource state: it is used to describe the security state of the resource entity in the service process. For example, the security status of the storage resource VM1 is safe.

System stability: it is used to describe the stability of the entire cloud service system, such as whether the cloud service behavior has a deadlock.

In order to describe these properties of security SLA, We present a SecSLA model based on TCTL, which can express the state ϕ required in QoS and the required state transition path, we use Backus-Naur Form to describe it as follows:

SecSLA:

```

SecSLA ::= Security: secSLA Performance: perfoSLA;
secSLA ::= data security: data_Sec virtual resource security: VR_Sec
data_Sec ::= BooleanMetric | operation
BooleanMetric ::= backup | access_control | data_encrypted | data_integrity
| ...
VR_Sec ::= BooleanMetric | operation
BooleanMetric ::= intruDetect | VulDetect | ...
operation ::= < StateSLA > | < StateSLA > < secSLA >;
StateSLA ::= < AtomSLA > | 'not' < StateSLA > | < StateSLA
>' (and | or | imply) ' < StateSLA > | 'A' < PathSLA > | 'E' <
PathSLA >;
AtomSLA ::= < ASID >' ' < State > | < EID >' ' < State
> | < ASID >' ' < SAttrName >< Compare >< Real >< EID >'
' < EAttrName >< Compare >< Real >;
Compare ::= ' <' | ' >' | ' <=' | ' >=';
PathSLA ::= < StateSLA >' U ' < StateSLA > | ' F ' < StateSLA
> | ' G ' < StateSLA >;
perfoSLA ::= ConditionPerformance | NumericMetrics
NumericMetrics ::= Memory | Delay | CPU | Bandwidth | Availability ...
ConditionPerformance ::= ' Time : ' < TimeSLA >' Reliability : ' <
ReliabSLA >' Cost : ' < CostSLA >;
TimeSLA ::= < secSLA >'>'< CC > | < perfoSLA >'>'< CC >, <
TimeSLA >;
ReliabSLA ::= < secSLA >< Compare >< Real[0;1] >< secSLA ><
Compare >< Real[0;1] >, < ReliabSLA >;
CostSLA ::= < perfoSLA >< Compare >< Real > 0 >< perfoSLA ><
Compare >< Real > 0 >, < CostSLA >;
Time ::= < Clocks >< Invar >< TGuard >< CReset >;
< Clocks > ::= ' Clocks : ' < Clock > *;
< Clock > ::= < String >;
< CC > ::= < Clock >< Compare >< Real > 0 > | < CC >' <' <
CC >;
< Invar > ::= ' Invariants : '(< State >'>'< CC >)*;
< TGuard > ::= ' Time Guard : '(< Trans >'>'< CC >)*;
< CReset > ::= ' Clock Reset : '(< Trans >'>'(< Clock >'>'< 0 >)*)*;
Reliability ::= ' Reliability : ' < Succ > *;
Succ ::= < Trans >'>'< Real[0;1] >;
Cost ::= ' Cost : ' < StateCost >, < TransCost >;
< StateCost > ::= (< State >'>'< Real > 0 >)*;
< TransCost > ::= (< Trans >'>'< Real > 0 >)*;

```

5 Verifying cloudsec approach of conformance checking

In this section, we use UPPAAL to check SecSLA conformance at the bottom of the cloud. UPPAAL is a tool for modeling, simulating and verifying real-time systems, which can specify behavioral properties in logic. We performed our verification through UPPAAL 4.0.

In order to prove that our conformance checking result is consistent with reality, we build a real cloud storage scenario and verify security properties through UPPAAL.

The experiments presented in this section were conducted in a cluster running on OpenStack (Pike), which involves five physical machines corresponding to the compute node, storage node and control node. See the hardware configuration in Table 1.

Table 1 Hardware configuration

Resource type	Hardware configuration	OS	Network
Compute resource	6GiB memory, Intel(R)	Ubuntu	10 Gbit
	Core(TM) i5-2400CPU @ 3.10GHz		Etherne
	8GiB memory, Intel(R)	Ubuntu	10 Gbit
	Core(TM) i5-6500CPU @ 3.20GHz		Etherne
Storage resource	12GiB memory, Intel(R)	Ubuntu	10 Gbit
	Core(TM) i5-3470 CPU @ 3.20GHz		Ethernet
Storage resource	16 GiB memory, Intel(R)	Ubuntu	10 Gbit
	Xeon(R) CPU i5-2420 @ 1.90GHz		Ethernet
Control resource	10 GiB memory, Intel(R)	Ubuntu	10 Gbit
	Core(TM) i5-3470 CPU @ 3.20GHz		Ethernet

Figure 5 illustrates the different topology of cloud storage service based on OpenStack. Various subnet virtual machines would provide different atomic cloud services. Figure 5(a) is a cloud service environment topology with uniform distribution of virtual resource entities. Figure 5(b) is a cloud service environment with nonuniform distribution in three different subnets, which can be used to simulate cloud services interacting with each other in different regions. Figure 5(c) is a random cloud environment topology with multiple virtual entities in different subnets, which is most similar to real cloud service scenario. We experiment and monitoring the security condition when the cloud provides data isolation, fair schedule, data storage and virtual machine migration behaviors in different cloud environment topology.

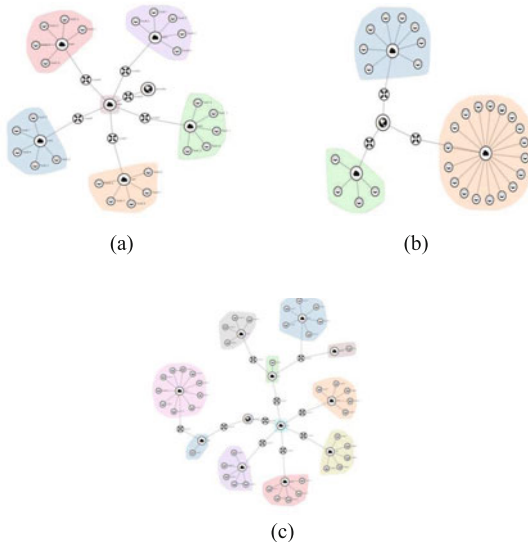
**Fig. 5** The Topology of OpenStack. (a) Uniform topology; (b) nonuniform topology; (c) random topology

Figure 6 specifically illustrates a detailed framework for cloud service security conformance verification, which includes a cloud service pool, an entity knowledge base, a security SLA model, and a UPPAAL verification module. The whole validating process can be divided into the following steps:

STEP1: Obtaining the real-time QoS metrics of the resource entity through the Cloud log, and building the entity knowledge base includes physical entities and virtual resource entities to describe the characteristics of the resource entity.

STEP2: Modeling various atomic service entities by CloudBeh model characterizes the state changes of an atomic service entity and its interaction with resource entities.

STEP3: Forming a composite service entity by combining atomic service entities and resource entities, and creating the cloud service pool with the atomic entity.

STEP4: Describing the cloud service QoS requirements by SecSLA based on TCTL to describe the user's cloud service security quality requirements.

STEP5: Determining whether the service behavior of the cloud satisfies the user's requirements of service security quality through UPPAAL and exporting the results.

Algorithm 2

Function *cloud storage service QoS conformance checking*

*/*Initialisation*/*

...

While *true* do

 Res $\leftarrow$ ReceiveQueriesResult()

 currentState $\leftarrow$ EvalCurrentState()

 [changed, pars] $\leftarrow$ EvalCloudParams(testbed_ OpenStack)

 If changed OR (CurrentState $\neq$ oldState) then

 SetCloudParams (currentState, pars)

 BuildModel ()

 End If

 secSLA $\leftarrow$ ModelCheckProperties (SecSLA_props)

 breach $\leftarrow$ Compare(secSLA)

 If (breach) then

/ nonconformance*/*

 Break

 End If

 Oldstate $\leftarrow$ CurrentState

End While

End function

5.1 Monitoring and correlating cloud log

In order to effectively validate cloud service conformance, the first key step is to obtain real-time QoS data and the status of cloud service behavior. So we assume that this information can be sent and received by service log in real time from

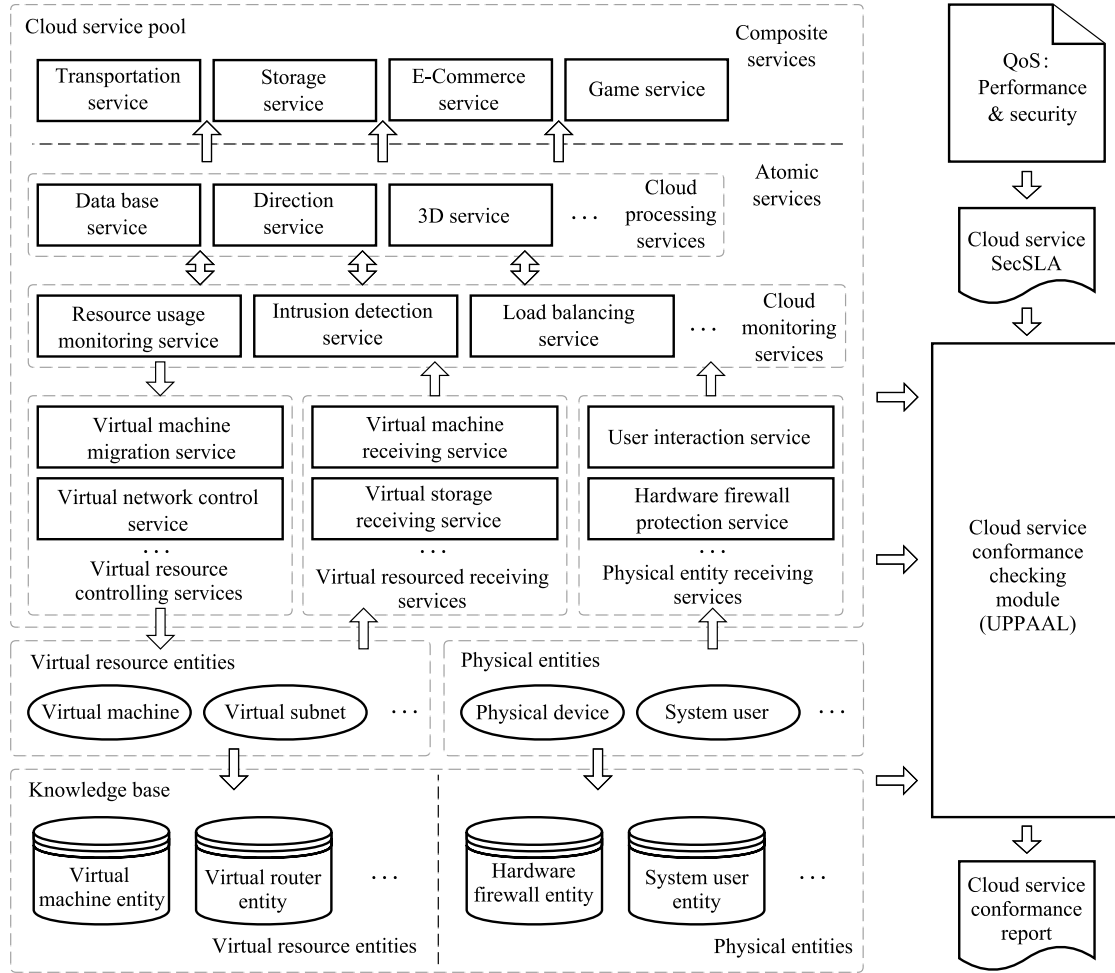


Fig. 6 The framework of cloud service security conformance checking in experiment

the OpenStack platform. Furthermore, we use Algorithm 2 to obtain virtual resources state transition to verify the conformance of cloud storage service in our testbed.

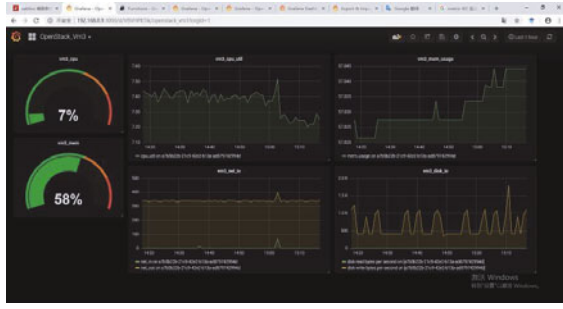
Figure 7 presented the monitoring resulting through Algorithm 2 based on OpenStack. We can obtain service information in real-time. For example, Fig. 7(a) shows different virtual machines status and performance like CPU usage, memory, hard disk, running time, etc. Figure 7(b) presents the status and performance of key node such as compute node, storage node. These metrics are essential when verifying security conformance.

5.2 Validating CloudSec approach

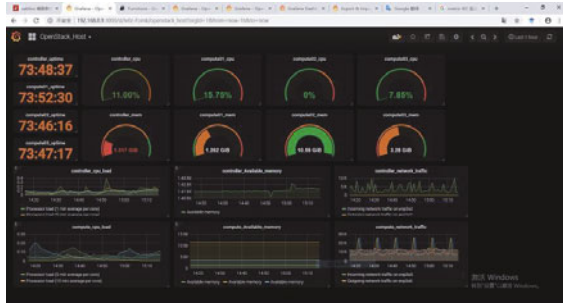
In experiment, four typical composite behaviors of cloud storage services are presented by CloudBeh model, namely Data isolation, Fair scheduling, Virtual machine migration, and Data storage. These four composite behavior are modeled by timed automata network discussed above. The depicted model is shown in Fig. 8. Each composite behavior

is modeled through atom behavior and interactions between them and resources.

- **Data isolation:** This behavior should ensure that storage resources are supplied by multiple virtual machines, but the data of different cloud users are isolated and would not be used by other users, and keeping data independence.
- **Fair scheduling:** This behavior is to ensure that different service resources of the cloud service can be fairly scheduled.
- **Data storage:** This behavior ensures that the most appropriate virtual resources are allocated for the different user according to the user's security QoS requirements for storing user data.
- **Virtual machine migration:** This behavior migrates data from a virtual machine to another virtual machine under specific time and security constraints, as guaranteeing data integrity.



(a)



(b)

Fig. 7 The monitoring resulting of OpenStack. (a) Fundamental topology monitoring; (b) multiple topology monitoring

Among these four behaviors, we can see that access control and data privacy are the most important security SLA entries. Each behavior must ensure that the data is not obtained and used by other unrelated users ensuring the privacy of the data.

In order to illustrate that this method can effectively verify the cloud service conformance and discover the risk of cloud service behavior, we use the random topology structure as the experimental environment to verify the access control and privacy of the four cloud composite services discussed above.

The experimental results are shown in Table 2. The Cloud-Sec method can effectively perform conformance verification and discover potential risks in the behavior of cloud services.

Firstly, we check the liveness of composite behavior, which means that the automata model have not to deadlock and our model is accurate to illustrate the state transition. Moreover, then we verify the access control, it is determined by checking whether a particular node can access a particular port. Privacy is also checked by our approach. We notice that there have some mistakes in virtual machine migration behavior, the security SLA as “vm1.backup $\rightarrow$ mi.migration.c $<$ 5, vm1.backup $\rightarrow$ vm2.backup.c $<$ 5 and $\exists$ vm2.safe” is checked with result can not be verified. We could analyze the simulation trace running in UPPAAL and find the cause of the mistake.

As Fig. 9 illustrates, we can conclude that when vm1 is in the backup state, although cloud processing service would take migration action, but due to cloud monitoring service get vm2 state is in danger, so cloud control service cannot switch

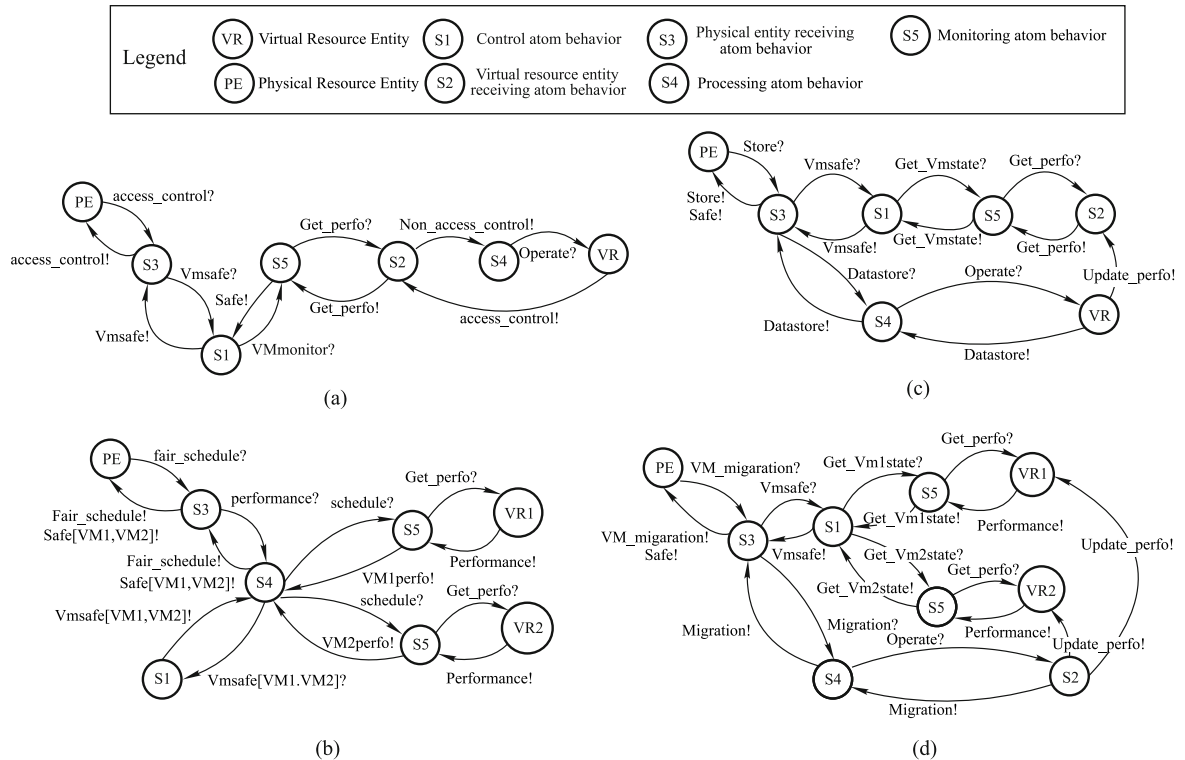


Fig. 8 Composite behavior in cloud storage. (a) Data isolation; (b) fair schedule; (c) data storage; (d) VM migration

Table 2 The experimental results in openstack

Cloud service behavior	Security constraints	Test	Monitor result	CloudSec approach
Data isolation	Access	User1 cannot access other subnet 80 port service	Not access	Not access
	Privacy	Data in different storage node is encrypted	Encrypted	Encrypted
Fair schedule	Access	User1 can access subnet 80 port service	Access	Access
	Privacy	User1 uses tcp to encrypted access service node	Access	Access
Data storage	Access	User1 can access specific storage node.	Access	Access
	Privacy	Storage node can process IDS	Success	Success
Virtual machine migration	Access	Control node 1 can access subnet 21 port service	Access	Access
	Privacy Backup	Storage node1 backup its data in storage node2	Backup	Non-Backup

vm2 to backup state, which causes the failure of the virtual machine migration backup. Through the analysis of the vm2, as Fig.10 showed, we find that there is not deployed intrusion detection strategy, which leads to vm2 state is in danger. When we redeploy the vm2 intrusion detection strategy, the properties of “vm1.backup $\rightarrow$ vm2.backup” and “ $\exists$ vm2.safe” are satisfied. So although the behavior of vm1 backup, its data in vm2 is operated in OpenStack but would have potential security issue threatening our cloud environment.

```
(danger, storage, danger, s1, s1, s1, s1)
showperfo: mi  $\rightarrow$  s1
(danger, storage, danger, s1, s1, s1, s1)
showperfo: mi  $\rightarrow$  s1
(danger, storage, danger, s1, s1, s1, s1)
vm2backup: vm2  $\rightarrow$  p2
(danger, storage, backup, s1, s1, s1, s2)
makevmlmigration: p2  $\rightarrow$  c1
(danger, storage, backup, s1, s3, s1, s3)
vmldanger: c1  $\rightarrow$  vm1
(danger, storage, backup, s1, s1, s1, s3)
showperfo: mi  $\rightarrow$  s1
(danger, storage, backup, s1, s1, s1, s3)
showperfo: mi  $\rightarrow$  s1
(danger, storage, backup, s1, s1, s1, s3)
showperfo: mi  $\rightarrow$  s1
(danger, storage, backup, s1, s1, s1, s3)
showperfo: mi  $\rightarrow$  s1
(danger, storage, backup, s1, s1, s1, s3)
vm2danger: vm2  $\rightarrow$  p2
(danger, storage, danger, s1, s1, s1, s1)
```

Fig. 9 The trace of virtual machine migration

```
(C) Copyright 2001-2002, Martin Roesch, Brian Caswell, et al.
# All rights reserved.
# $Id$
# DOOS RULES
#
alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"DOOS Trn Probe"; id: 678; type: 8; content:
43; classtype:attempted-recon; sid:221; rev:1);
alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"DOOS TrnK icmp possible communication"; type:
AAAAAAAA; reference:armitage,425; classtype:attempted-dos; sid:222; rev:1);
alert udp $EXTERNAL_NET any -> $HOME_NET 31335 (msg:"DOOS Trn000:Daemontomaster(PONGdetected)";
threshold:10; classtype:attempted-recon; sid:223; rev:1);
```

Fig. 10 The monitor result of VM2

whether cloud service behavior meets the security QoS requirements. This approach is helpful to discover the potential risks of cloud services, find out the causes of problems by analyzing the state service behavior state transition, and improve the security service quality.

The above discussion focuses on the verification of our approach in random topology environment, we would need to illustrate that this method is also effective for different cloud environments, we verify the six types of properties specified in the SecSLA under different network topologies. The accuracy of the method is discussed in Section 5.3 in detail.

5.3 Comparison experiment

In order to illustrate our approach is more effectiveness in cloud security conformance checking, We would compare CloudSec approach with other three types of related work, respectively are based on numeric SLA, service process, and model checking. We select typical methods discussed in [6, 8, 14] corresponding to the above three aspects, named them A1, A2 and A3 separately, and our approach is A4.

We also use above cloud storage scenario, and verify the different type of security SLA term, which are discussed as QoS values(P1), reachability(P2), time constraints(P3), interactivity(P4), resources state (P5)and system stability(P6), and give some cases to illustrate in Table 3.

Through Table 3, we can see that for the property P1, all methods can effectively verify the numeric SLA entries. This is because all conformance verification methods take into account numerical constraints. For the property P2, for checking the reachability of the state, the methods A1 and A2 cannot be effectively checked, this is because method A1 only focuses on numerical indicators and cannot determine the status of the service, while A2 pays more attention to the order of the services, and can only judge the service status corresponding to the process, other states cannot be accurately determined. “AG (Storage_VM1.FAULT imply AF Controlling_VM1.on) $\rightarrow$ (c < 5)” is an interaction

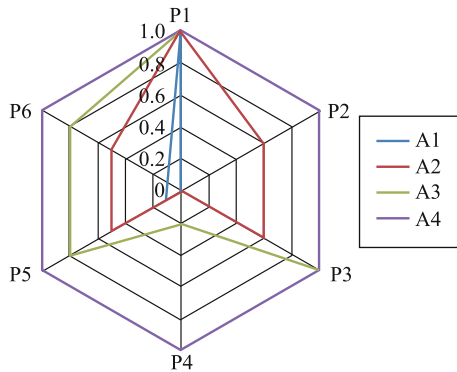
Therefore, the CloudSec method can effectively check

Table 3 The security conformance checking results in different approaches

SecSLA	A1: Approach based on QoS values [14]	A2: Approach based on service process [6]	A3: Approach based on model checking [8]	A4: CloudSec approach
P1: VM1.usability > 0.9	√	√	√	√
P2: A G (not (Storage_VM1.FAULT and Storage_VM2.FAULT)) > 0.9	×	√	√	√
P3&P4: A G (Storage_VM1.FAULT imply A F Controlling_VM1.on) → (c < 5)	√	×	√	√
P5: VM1.storage > 1000 → vm1.backup	×	√	√	√
P6: access_control_VM1.FAULT → Controlling_VM1.intruDetect	×	×	×	√

property that combines the P3 and P4 to control different service states under time constraints, so the A1 and A2 methods cannot be detected because it needs to describe the interaction and time constraints of the service. P5 describes the virtual resource security state, since the A2 method describes the service process, it is possible to determine some P5 properties which have the security state related to the process. “access_control_VM1.FAULT → Controlling_VM1.intruDetect” is a property that checks the security conformance of the overall cloud service system. This type of P6 can only be detected by the method given in this paper because CloudSec can detect the security conformance of the composite service behavior, which contains atom behavior and multiple resources.

Based on the above discussion, we repeated the experiments based on different cloud environment topologies for many times in order to calculate the conformance checking capabilities of the four methods for six different properties, Fig. 11 shows result, it indicates that CloudSec approach would be more fine granularity in cloud behavior modeling and more suitable using for cloud security SLA conformance checking, the details would be analyzed in next section.

**Fig. 11** The conformance checking ability between different approach

5.4 Results discussion

In order to verify the effectiveness of CloudSec approach,

we randomly generate the experimental cloud scenario just like Fig. 5 showed, with different scale of cloud service. Then CloudSec approach (A4) and other three comparison approaches (A1, A2 and A3) are implemented for verifying six different types of SLA term (P1, P2, P3, P4, P5 and P6), and each validation of the same category is repeated 100 times.

The effectiveness results are shown in Fig. 12. From (a)-(f), x-axis represents the scale of the cloud scenario and y-axis represents the effectiveness of each approach. The results of A4 are far better than A1, A2 and A3. With the cloud resources increasing, the A1 results do not decrease as it reaches the lower limit, while the other three approaches results decrease steadily, as there are more excellent effectiveness for various SLA category.

A1 approach based on QoS values [14] which solves the problem about $Results = checkconformance(< CS, VR >, QoS_values)$, it only focuses on simple QoS values of one VM or specific atom service, but cannot be used to check the QoS involved in different VR or service, which is caused by the interaction between them. As Fig. 12 showed, A1 can successfully detect the P1 like as “VM1.storage > 1000”, but cannot judge another type SLA like as P4, “VM1.storage > 1000 → vm1.backup”.

A2 approach based on service process [6] which solves the problem about $Results = checkconformance(Process_{cs}, performanceQoS)$, it aims to verify the cloud service process behaved as expected within some performance QoS constraints. However it ignores the mutual influence between different atom service or VM as P4, P5, and P6. For example, we can formalize the VM1’s process in the cloud, so the term of P2 like as “VM1.storage > 1000 → vm1.backup” can be easily checked, but if we want to check the term which is included in different atom service like P5, as “access_control_VM1 → Controlling_VM1.intruDetect”, it is not available.

A3 approach based on model checking discussed in [8]

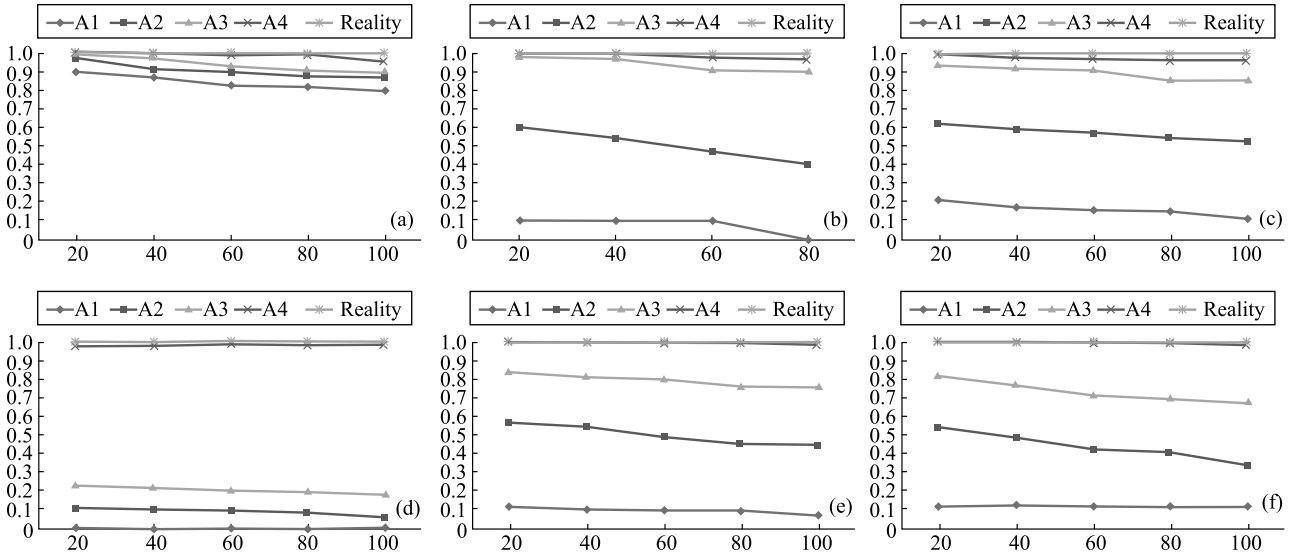


Fig. 12 The execution results of different approach. (a) P1; (b) P2; (c) P3; (d) P4; (e) P5; (f) P6

solves the problem about $Results = checkconformance(PSM_{CS}, SecSLA)$, which formalized the cloud service behavior as a probabilistic state machine which can express the state transition probability in cloud service. Through this approach, we can check the security conformance of one VM or cloud service and obtain the term related to probability. As an example, P5 like as “A G (not (Storage_VM1.FAULT and Storage_VM2.FAULT)) > 0.9” can be easily checked, another P5 property which is not considered probabilistic like as “access_control_VM1 $\rightarrow$ Controlling_VM1.intruDetec” also cannot be detected because it represents implication relationship between two different atom service.

So our approach A4 has advantage of expressiveness and fine-grained description in cloud service modeling, it can represent the interaction between atom cloud service and resources entities, also considering the constraints of each behavior, the superiority of A4 will be more effectiveness when the scale of cloud scenario increases especially in complex cloud environment, which can maintain high accuracy in cloud conformance checking.

6 Conclusion

In this paper, we propose an innovative CloudSec framework, which can effectively solve the opaque problem between users and the cloud services, and help users check whether the cloud services they use meet their security SLA requirements and whether the user’s data is securely stored and guaranteed, enhance user’s trust about cloud services. The approach gives a fine-grained cloud service modeling

model, called CloudBeh Model, and a SecSLA model that can describe all kinds of security entries. The model checking thoughts are used to implement security SLA conformance verification for complex cloud services. Finally, the effectiveness of the method is verified on the actual OpenStack platform. However, our approach also has some limitations, the state explosion which is the traditional problem of FSM also has affected in our approach, we try to divide cloud service in finer granularity atom service in order to reduce its influence.

Acknowledgements This work was supported by the National Natural Science Foundation of China (Grant Nos. U1636208, NO 61862008).

References

1. Krutz R L, Vines R D, Brunette G. Cloud Security: A Comprehensive Guide to Secure Cloud Computing. Indianapolis: Wiley, 2010
2. Singh A, Chatterjee K. Cloud security issues and challenges: a survey. Journal of Network and Computer Applications, 2017, 79: 88–115
3. Lomuscio A, Penczek W, Solanki M, Szreter M. Runtime monitoring of contract regulated web services. Fundamenta Informaticae, 2011, 111(3): 339–355
4. Van der Aalst W, Adriansyah A, van Dongen B. Replaying history on process models for conformance checking and performance analysis. Wiley Interdisciplinary Reviews Data Mining and Knowledge Discovery, 2012, 2(2): 182–192
5. Van der Aalst W M P, Dumas M, Ouyang C, Rozinat A, Verbeek E. Conformance checking of service behavior. ACM Transactions on Internet Technology, 2008, 8(3): 1–30
6. García-Banuelos L, Van Beest N R T P, Dumas M, La Rosa M, Mertens W. Complete and interpretable conformance checking of business processes. IEEE Transactions on Software Engineering, 2015, PP(99):

1–1

7. Kai J, Miao H, Zhao K, Zhou J, Gao H. A platform for analyzing behaviors of service-oriented application based on the probabilistic model checking. *International Journal of Software Innovation*, 2015, 3(2): 24–38
8. Kikuchi S, Aoki T. Evaluation of operational vulnerability in cloud service management using model checking. In: *Proceedings of IEEE International Symposium on Service Oriented System Engineering*. 2013, 37–48
9. Moreno G A, Camara J, Garlan D, Schmerl B. Proactive self-adaptation under uncertainty: a probabilistic model checking approach. In: *Proceedings of Joint Meeting on Foundations of Software Engineering*. 2015, 1–12
10. Leemans S J J, Fahland D, Van der Aalst W M P. Scalable process discovery and conformance checking. *Software and Systems Modeling*, 2018, 17(2): 1–33
11. Liu D, Zic J. Cloud#: a specification language for modeling cloud. In: *Proceedings of IEEE International Conference on Cloud Computing*. 2011, 533–540
12. Uriarte R B, Tiezzi F, Nicola R D. SLAC: a formal service-level-agreement language for cloud computing. In: *Proceedings of IEEE/ACM International Conference on Utility and Cloud Computing*. 2014, 419–426
13. Bergmayr A, Wimmer M, Kappel G, Grossniklaus M. Cloud modeling languages by example. In: *Proceedings of IEEE International Conference on Service-oriented Computing and Applications*. 2014, 137–146
14. Cicotti G, Coppolino L, D'Antonio S, Romano L. Runtime model checking for SLA compliance monitoring and QoS prediction. *JoWUA*, 2015, 6(2): 4–20
15. Awad A, Decker G, Weske M. Efficient compliance checking using BPMN-Q and temporal logic. In: *Proceedings of International Conference on Business Process Management*. 2008, 326–341
16. Hendre A, Joshi K P. A semantic approach to cloud security and compliance. In: *Proceedings of IEEE International Conference on Cloud Computing*. 2015, 1081–1084
17. Khadraoui A, Feltus C. Service specification and service compliance: how to consider the responsibility dimension? *Journal of Service Science Research*, 2012, 4(1): 123–142
18. Madi T, Majumdar S, Wang Y, Pourzandi M, Debbabi M, Jarraya Y, Wang L. Auditing security compliance of the virtualized infrastructure in the cloud: application to openstack. In: *Proceedings of the 6th ACM Conference on Data and Application Security and Privacy*. 2016, 195–206
19. Darvas D, Majzik I, Vinuela E B. Conformance checking for programmable logic controller programs and specifications. In: *Proceedings of the 11th IEEE Symposium on Industrial Embedded Systems*. 2016, 1–8
20. Roehm H, Oehlerking J, Woehrle M, Althoff M. Reachset conformance testing of hybrid automata. In: *Proceedings of the 19th International Conference on Hybrid Systems: Computation and Control*. 2016, 277–286
21. Viriyasitavat W, Da Xu L, Viriyasitavat W. Compliance checking for requirement-oriented service workflow interoperations. *IEEE Transactions on Industrial Informatics*, 2014, 10(2): 1469–1477



Xiaochen Liu is currently a PhD candidate at School of Computer Science and Engineering in Beihang University, China. She received the BS degree and the MS degree in school of Computer Science and Technology in Beijing Posts and Telecommunications University, China. Her research interests include network management, cloud service management, and cloud security analysis.



Chunhe Xia is now heading the Beijing Key Laboratory of Network Technology, Beihang University, China. He received his PhD degree in Computer Science and Engineering from Beihang University, China in 2003. His research interests include network security, network management, and network measurement.



Tianbo Wang received the PhD degree in computer application from the Beihang University, China in 2018. He is currently a lecturer with Beihang University, China. He has participated in several National Natural Science Foundations and other research projects. His research interests include network and information security, intrusion detection technology, and information countermeasure.



Li Zhong received the BS degree in computer science from Beihang University, and is currently a PhD student of Beihang University, China. His mainly research includes cloud computing technology, social network analysis and source identification of information diffusion.



Xiaojian Li received his PhD degree in Computer Science from Beihang University, China. He is a professor in the School of Computer Science and Information Technology at Guangxi Normal University, China. His research interests focus on the field of cyberspace, including computer network, concealed communication, network penetration, intelligence analysis, and information system security.